Walchand College of Engineering, Sangli

(Government Aided Autonomous Institute)

7CS353 Machine Learning Lab — Assignment 1

Python Fundamentals Through Your First Dataset

Class, Semester	Third Year B. Tech. (CSE), Sem V — AY 2025-26
Linked courses	7CS353 (Lab) and 7CS302 Module I & II (Foundational Maths, EDA)
Course outcome	CO1 — use Python fundamentals and relevant libraries for applying EDA and ML techniques on a distribution or dataset (Apply)
Duration	1 lab session (2 hrs) + take-home challenge; submit within 1 week
Assessment	Counts toward LA1 (lab activity + journal); viva during next session
Prerequisites	None. No prior Python experience is assumed.

1. Why this assignment

Machine learning is the art of teaching a computer to find patterns in data. Before any algorithm, you need two skills: writing simple Python, and looking at a dataset the way a data scientist does. In this assignment you will build both at once. Every Python concept is introduced through a small dataset of student exam records — the same dataset your theory course will use to illustrate vectors, distributions, and mean/variance in Module I.

2. Objectives

- Set up and use a Python environment (Python with IDE or Jupyter Notebook) without any installation headaches.
- Use variables, lists, loops, conditionals, and functions — the minimum Python needed for ML.
- Represent data as vectors using NumPy and compute mean, variance, and standard deviation.
- Load a CSV with pandas, summarize it, and draw your first histogram and scatter plot.

3. Environment setup

ANY ONE of following environmental set up is recommended:

1. Install Python (latest stable version) and any of your favourite IDE VS Code / PyCharm
2. Install Anaconda (anaconda.com) and open Jupyter Notebook or Spyder

Note: Use of Google Colab is not encouraged at the initial stage.

Instructions for running the code:

1. Get acquainted with notebook style of coding with Jupyter of with (#%% in VS Code)
2. Each box is a "cell": type code, press Shift+Enter to run it. Verify your setup by running:

```
print("Hello, Machine Learning!")
```

3. Understand simple import statements in python

4. The dataset

Write below code in a cell to create the dataset used throughout the assignment. It records marks of 12 students in two tests plus their attendance percentage.

```
names      = ["Aarav","Diya","Ishaan","Meera","Kabir","Anaya",
              "Rohan","Sara","Vivaan","Priya","Arjun","Nisha"]
test1      = [56, 78, 45, 88, 62, 91, 40, 73, 67, 85, 52, 95]
test2      = [61, 82, 50, 90, 58, 94, 38, 70, 72, 88, 47, 97]
```

```
attendance = [72, 88, 65, 95, 70, 98, 55, 80, 78, 92, 60, 99]
```

5. Part A — Python fundamentals

Type and run each snippet, then complete the task. Do not copy-paste: typing builds fluency and avoids copying of Unicode characters.

A1. **Variables and printing.** Run:

```
topper = "Nisha"
marks = 95
print(topper, "scored", marks, "in Test 1")
```

Task: create variables for your own name and expected marks, and print a similar sentence. Explore 'Output Formatting in Python'.

A2. **Lists and indexing.** Python counts from 0. Run:

```
print(test1[0])      # first student's marks
print(test1[-1])     # last student's marks
print(len(test1))    # how many students
```

Task: print the name and Test 1 marks of the 5th student in one line.

A3. **Loops.** Print every student with their marks:

```
for i in range(len(names)):
    print(names[i], test1[i])
```

Task: modify the loop to also print Test 2 marks and the improvement (test2 minus test1) for each student. Explore 'enumerate' keyword in python.

A4. **Conditionals.** Task: using a loop and an if statement, print only the names of students who scored more than 80 in Test 1, and count how many such students there are.

A5. **Functions.** Complete this function so it returns the average of any list, then use it on test1, test2, and attendance:

```
def average(values):
    total = 0
    # your code: add up all values, divide by how many
    return total
```

Checkpoint question (write in a markdown cell): the average of Test 2 should come out higher than Test 1. What does that tell you about the class as a whole? Can the average alone tell you whether every student improved?

6. Part B — Data as vectors with NumPy

Each data point is a vector and a dataset lives in a vector space. NumPy is how Python does vectors. Here each student is a point in 3-dimensional space: (test1, test2, attendance).

B1. **From lists to vectors.** Run:

```
import numpy as np
t1 = np.array(test1)
t2 = np.array(test2)
att = np.array(attendance)
print(t2 - t1)   # improvement of ALL students in one line — no loop!
```

Notice: what needed a loop in Part A is one line with vectors. This "vectorized" style is how all ML libraries work.

B2. **Summary statistics.** Compute and print the mean, variance, and standard deviation of Test 1 using np.mean, np.var, np.std. Verify that np.mean(t1) matches your own average() function from Part A.

B3. **Interpreting spread.** Compute the standard deviation of Test 1 and of attendance. Which is more "spread out"? Write one sentence in your journal on what a large standard deviation means about the class.

B4. **A first taste of geometry.** Each student is a vector. Compute the distance between two students:

```
meera  = np.array([88, 90, 95])
kabir  = np.array([62, 58, 70])
dist = np.sqrt(np.sum((meera - kabir)**2))
print(dist)
```

Task: find which student is "closest" to Nisha in this 3-D space. (Hint: loop over all students, compute the distance, track the minimum — skip Nisha herself.)

7. Part C — First look at real EDA with pandas

Real data comes in files, not typed lists. Create a CSV once by running:

```
import pandas as pd
df = pd.DataFrame({"name": names, "test1": test1,
                   "test2": test2, "attendance": attendance})
df.to_csv("marks.csv", index=False)
```

C1. **Load and inspect.** Run df = pd.read_csv("marks.csv"), then try df.head(), df.shape, df.describe(). In a markdown cell, note what count, mean, std, min, 25%, 50%, 75%, max mean in the describe() output.

C2. **Selecting and filtering.** Print only the name and test1 columns; then print all rows where attendance is above 85 using df[df["attendance"] > 85].

C3. **Histogram — your first distribution.** Run:

```
import matplotlib.pyplot as plt
plt.hist(df["test1"], bins=5)
plt.xlabel("Test 1 marks"); plt.ylabel("Number of students")
plt.show()
```

Expected observations (write in markdown cell): does the shape look roughly bell-shaped? Which distribution it resembles to?

C4. **Scatter plot — spotting a relationship.** Plot attendance on the x-axis vs test1 on the y-axis using plt.scatter. Journal question: do students with higher attendance tend to score more? Could you draw a straight line through the cloud of points? (That line is linear regression — Module III.)

8. Part D — Take-home challenge

D1. Write a function grade(marks) that returns "A" for 85 and above, "B" for 70–84, "C" for 50–69, and "F" below 50. Add a new column "grade" to the DataFrame for Test 2 and print how many students got each grade (hint: df["grade"].value_counts()).

D2. One student's Test 1 entry was mistyped: change Rohan's test1 to 400 and recompute the mean and standard deviation. Write 2–3 sentences on how a single wrong value distorted both numbers.

D3. Optional stretch: without using np.var, compute the variance of test1 yourself using only a loop and the formula: average of squared differences from the mean. Verify it matches np.var(t1).

9. Deliverables and submission guidelines

- One notebook (.ipynb) named RollNo_Name_Assignment1.ipynb containing all parts, with each task in its own cell and outputs visible.
- Observations in markdown cells. Answers to all checkpoint/ questions in your own words.
- Make a word document with title of the assignment, PRN, name of the student and date. Paste screenshots of task-wise output and observations.
- Make a folder with name PRN_MLL_A1. Include your source files (.py or .ipynb), any dataset file used in the assignment and the word document. Zip the folder and submit on ERP submission link within stipulated timeline.
- A short evaluation viva would be conducted after the submission.

10. Assessment rubric (10 marks, toward LA1)

Component	Marks	What is checked
Part A — Python basics (tasks A1–A5)	3	Code runs; loop/condition/function written by the student
Part B — NumPy vectors & statistics	2	Correct mean/variance; nearest-student task attempted
Part C — pandas & plots	2	CSV loaded; histogram and scatter with labelled axes
Part D — challenge	2	Grade function correct; outlier reflection is thoughtful
QnA / Viva	1	Concepts explained in own words

AI generated code leads to 0 marks.

11. Rules and help

- Type all code yourself. Discussing with classmates is encouraged; submitting identical notebooks is not.
- Using AI tools to generate answers defeats the purpose of this fluency-building assignment.
- Stuck? Read the error message, check spelling and indentation (Python is strict about both), then ask the lab instructor.
- Recommended supplements:
  1. "Python for Everybody" (Ch. 1–5, free online)
  2. StatQuest videos on mean/variance
  3. Essence of calculus and linear algebra: https://www.youtube.com/watch?v=WUvTyaaNkzM